
Heart Disease Prediction System

Project Id: AIML-13

Name : Rishika Kapoor (Team Leader)

Class Roll Number: 22

University Roll Number : 2028476

Section and Group: U(G1)

Name : Aakshi

Class Roll Number: 02

University Roll Number : 2028375

Section and Group: U(G1)

Name : Riddhima Gairola

Class Roll Number: 20

University Roll Number : 2028473

Section and Group: U(G1)

TABLE OF CONTENTS

1. Problem Description	3
1.1.Dataset Description.....	3
1.2.Flow diagram.....	4
2. Modules for Project Implementation.....	5
2.1. Modules Used.....	6
2.2. Platform Use.....	6
3. Machine Learning model/ Searching Technique used	7
4. Evaluation metrics used..	8
5. Results and Visualizations	9
6. Conclusion and Future Scope	14
7. References.....	15

1. Problem Description

Heart disease is one of the leading causes of death worldwide. Early detection and prediction of heart disease can help doctors take preventive measures and provide timely treatment to patients. However, analyzing large amounts of medical data manually can be difficult and time-consuming. Machine Learning techniques can help in identifying patterns in medical data and predicting whether a person is at risk of heart disease. The aim of this project is to design and develop an end-to-end Machine Learning classification system that predicts the presence of heart disease using clinical patient data. The system will use a heart disease dataset containing medical attributes such as age, blood pressure, cholesterol level, chest pain type, and heart rate. The dataset will be preprocessed by handling missing values, normalizing numerical data, and converting categorical variables into numerical form. Different Machine Learning models such as Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Extra Trees Classifier and Gaussian Naive Bayes will be trained on the dataset. The performance of these models will be evaluated using metrics like accuracy, precision, recall, F1-score, confusion matrix, and ROC curve. The final model will help in predicting whether a patient has heart disease (1) or not (0).

1.1 Dataset Description

The dataset comprises 50,000 records. Our analysis reveals that 46.3% of the individuals in this study have heart disease. The data shows that clinical factors like **high blood pressure**, **older age**, **high cholesterol**, and **diabetes** are the biggest indicators of heart health issues. Lifestyle choices also play a major role, as people who smoke or have a sedentary (inactive) lifestyle are much more likely to be diagnosed. While the data is very detailed, it is missing a lot of information regarding how much alcohol the individuals consume, which is something to keep in mind for further study. Given the strong correlations found, a machine learning model could likely predict heart disease in this dataset with high accuracy.

1.2 Flow Diagram

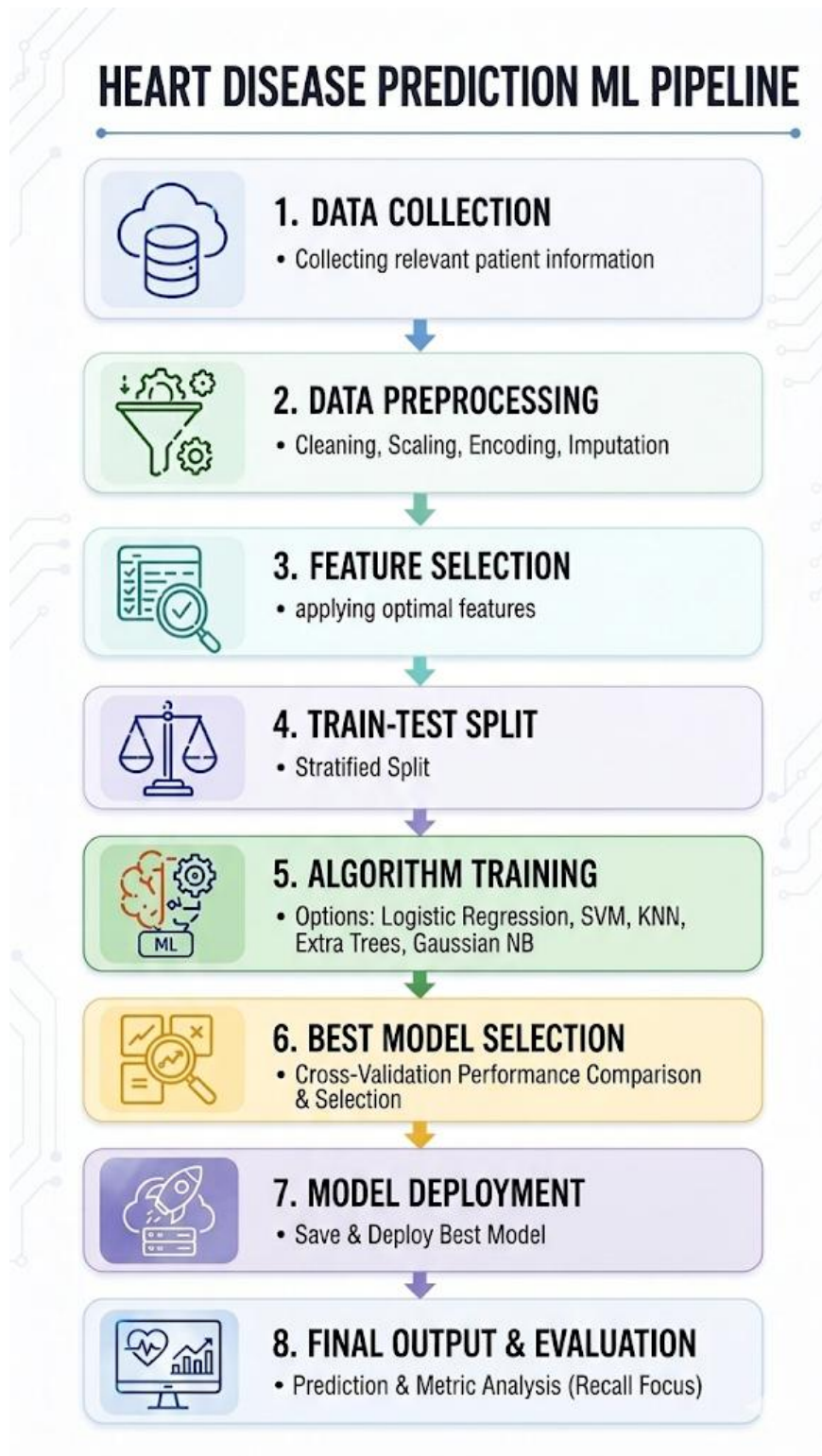


Fig.1 Flow diagram.

2. Modules for Project Implementation

2.1 Modules Used:

1. Pandas (import pandas as pd)

- **Purpose:** A powerful library used for data manipulation, cleaning, and analysis using DataFrame structures.
- **Functions used:**
 - `pd.read_csv()`: To load the raw dataset from a CSV file into a Pandas DataFrame.
 - `df.head()`: To display the first few rows of the dataset for an initial visual inspection.
 - `df.info()`: To retrieve a concise summary of the DataFrame, including data types and memory usage.
 - `df.describe()`: To generate descriptive statistics (mean, min, max, standard deviation) for the numerical columns.
 - `df.isnull().sum()`: To check for and count any missing or null values in the dataset.

2. NumPy (import numpy as np)

- **Purpose:** The fundamental package for scientific computing in Python, providing support for multi-dimensional arrays and high-level mathematical functions.
- **Functions used:**
 - Used generally for background mathematical operations and handling numerical arrays generated during machine learning model processing.

3. Matplotlib (import matplotlib.pyplot as plt)

- **Purpose:** A comprehensive library for creating static, animated, and interactive visualizations in Python.
- **Functions used:**
 - `plt.rcParams['figure.figsize']`: To globally configure the default width and height of the generated plots.
 - `plt.figure()`: To create a new figure or plotting area.
 - `plt.title()`, `plt.xlabel()`, `plt.ylabel()`: To add descriptive text and labels to the charts.
 - `plt.show()`: To render and display the final visualizations.

4. Seaborn (import seaborn as sns)

- **Purpose:** A statistical data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.
- **Functions used:**
 - `sns.heatmap()`: Used to plot the correlation matrix and visualize the confusion matrices with color-coded gradients.

5. Scikit-Learn (import sklearn)

- **Purpose:** The primary machine learning library used for predictive data analysis, model building, and evaluation.
- **Sub-modules and Functions used:**
 - **sklearn.model_selection:**
 - `train_test_split()`: To divide the dataset into training (for model learning) and testing (for model evaluation) subsets.
 - **sklearn.preprocessing:**
 - `StandardScaler()`: To standardize features by removing the mean and scaling them to unit variance.
 - `.fit_transform()`: To calculate the scaling parameters and apply them to the training data.
 - `.transform()`: To apply the previously calculated scaling to the testing data.
 - **sklearn.metrics:**
 - `accuracy_score()`: To calculate the overall percentage of correct predictions.
 - `classification_report()`: To generate a detailed text report showing the precision, recall, and F1-score for each crop.
 - `confusion_matrix()`: To compute a matrix evaluating the accuracy of a classification by showing True vs. False predictions.
 - **Machine Learning Algorithms (Models):** `LogisticRegression()`, `KNeighborsClassifier()`, `SVC()`, `DecisionTreeClassifier()`, `RandomForestClassifier()`, `ExtraTreesClassifier()` and `GaussianNB()`.
 - `.fit(X_train, y_train)`: The universal Scikit-learn function used to train all the aforementioned models on the data.
 - `.predict(X_test)`: Used by the trained models to generate crop recommendations based on the unseen testing data.

6. Warnings (import warnings)

- **Purpose:** Used to manage warning messages generated by Python or external libraries during execution.
- **Functions used:**
 - `warnings.filterwarnings('ignore')`: To suppress non-critical system warnings (such as future deprecation notices), keeping the Jupyter Notebook outputs clean and readable.

2.2 Platform Used:

- **Jupyter:** Jupyter Notebook is an open-source web application and interactive computing environment that allows users to create and share documents containing live code, visualizations, and narrative text. It served as the primary Integrated Development Environment (IDE) for building and testing our machine learning pipeline.

3. Machine Learning Model/ Searching Technique Used:

- **Logistic Regression:** Logistic Regression is a statistical algorithm used for classification tasks. It works by applying a logistic (sigmoid) function to a linear combination of the input features .
- **K-Nearest Neighbors (KNN):** KNN is a non-parametric, instance-based learning algorithm. Rather than building a mathematical equation during training, it stores the dataset and makes predictions by calculating the distance (typically Euclidean distance) between a new, unseen data point and all existing points.
- **Support Vector Machine (SVM):** SVM is a powerful algorithm that maps data points into a high-dimensional space to discover the optimal "hyperplane"—the mathematical boundary that provides the maximum margin of separation between different crop classes.
- **Extra Trees Classifier (Extremely Randomized Trees):** Closely related to the Random Forest, the Extra Trees classifier is also an ensemble method that builds multiple decision trees. However, it introduces a higher level of randomness: instead of mathematically calculating the most optimal threshold to split the data, it selects the splitting thresholds completely at random.
- **Gaussian Naive Bayes (Gaussian NB):** Gaussian Naive Bayes is a highly efficient, probabilistic classification algorithm grounded in Bayes' Theorem. The "Gaussian" aspect indicates that it assumes the continuous input environmental features follow a normal, bell-curve distribution.
- **Grid Search Cross-Validation (GridSearchCV):** Grid Search is an exhaustive, systematic searching technique. We provided the algorithm with a "grid" or a specific list of possible settings for a model (for example, testing different 'K' values in KNN or different 'kernels' in SVM). The Grid Search algorithm systematically tests every single possible combination of these parameters, evaluates the model's accuracy for each combination using cross-validation, and ultimately returns the exact mathematical combination that yields the highest performance.

4. Evaluation metrics used:

- a) **Accuracy:** This is the foundational metric of our evaluation. It calculates the overall correctness of the model by dividing the number of correct predictions (both True Positives and True Negatives) by the total number of predictions made.

$$Accuracy = (TP+TN)/(TP+TN+FP+FN)$$

- b) **Precision:** Precision measures the accuracy of positive predictions. High precision indicates a low false-positive rate.

$$Precision = TP / (TP + FP)$$

- c) **Recall (Sensitivity):** Recall measures the model's ability to find all relevant instances within a dataset.

$$Recall = TP / (TP + FN)$$

- d) **F1-Score:** The F1-Score is the harmonic mean of Precision and Recall. We consider this metric because it provides a single, balanced score that accounts for both false positives and false negatives.

$$F1 = 2 * (Precision * Recall) / (Precision + Recall)$$

- e) **Confusion Matrix:** While not a single mathematical score, the Confusion Matrix is a critical visual evaluation tool we utilized. It provides a detailed table showing exactly where a model is making errors by mapping the predicted classes against the actual classes.

- f) **ROC Curve (Receiver Operating Characteristic):** The ROC curve is a graphical representation of a model's performance across all possible classification thresholds. It plots the True Positive Rate (how many actual positives were correctly identified) against the False Positive Rate (how many negatives were incorrectly labeled as positive).

- g) **AUC (Area Under the Curve):** AUC is the single numerical value that summarizes the ROC curve. It measures the entire two-dimensional area underneath the entire ROC curve, providing an aggregate measure of performance. The score ranges from 0 to 1, where 1.0 represents a perfect model that makes zero mistakes.

5. Results and Visualizations

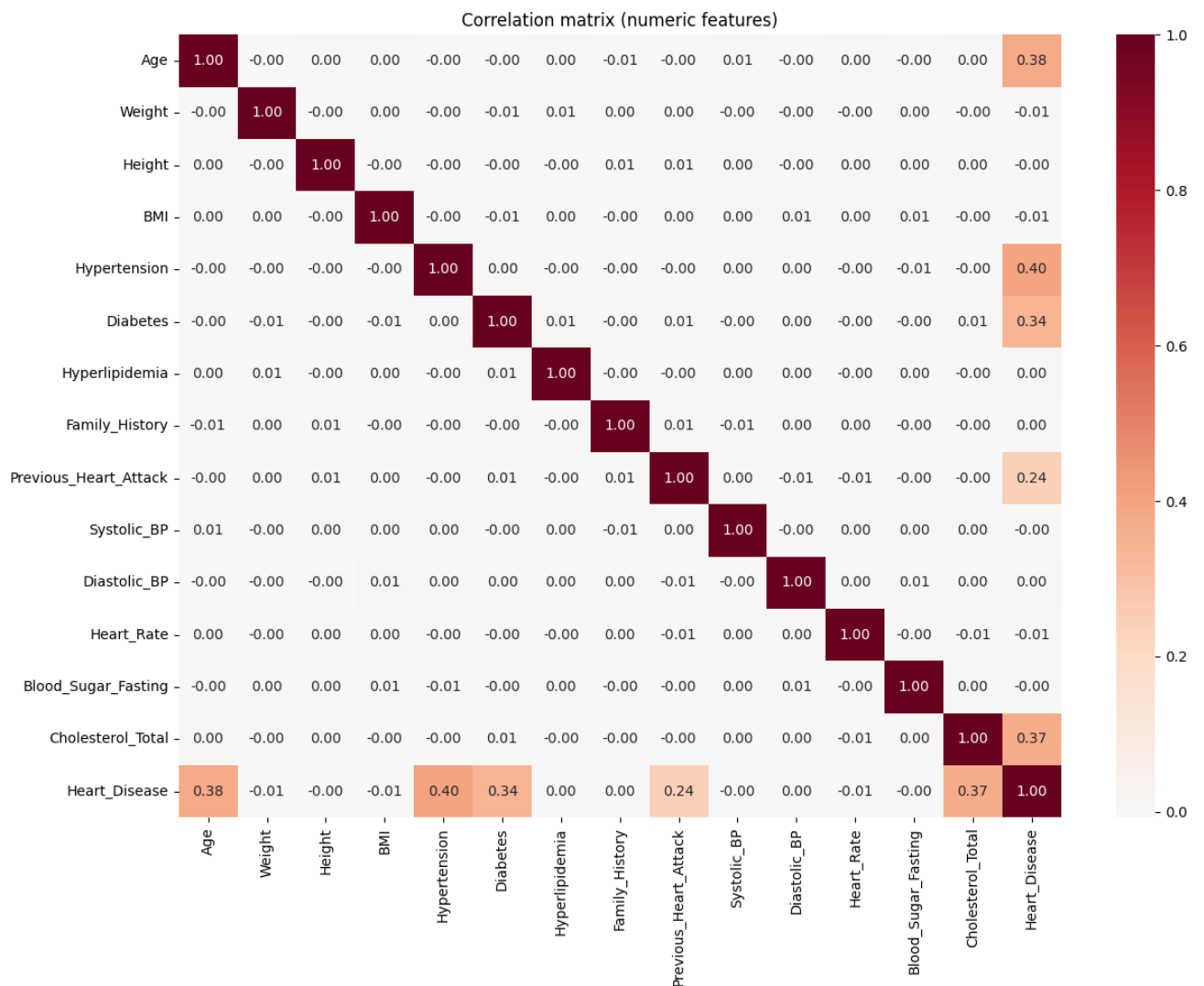


Fig.2 Heatmap showing correlation.

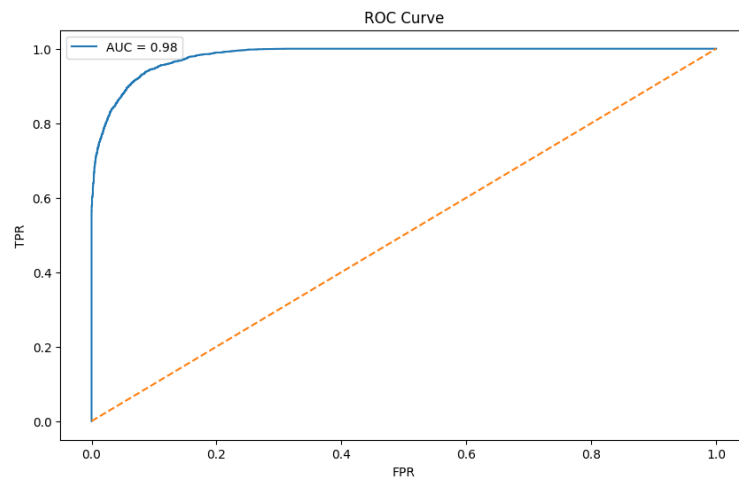


Fig.3 ROC (Logistic Regression)

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.93	0.93	5365
1	0.92	0.92	0.92	4635
accuracy			0.92	10000
macro avg	0.92	0.92	0.92	10000
weighted avg	0.92	0.92	0.92	10000

Fig.4 Classification Report(Logistic Regression)

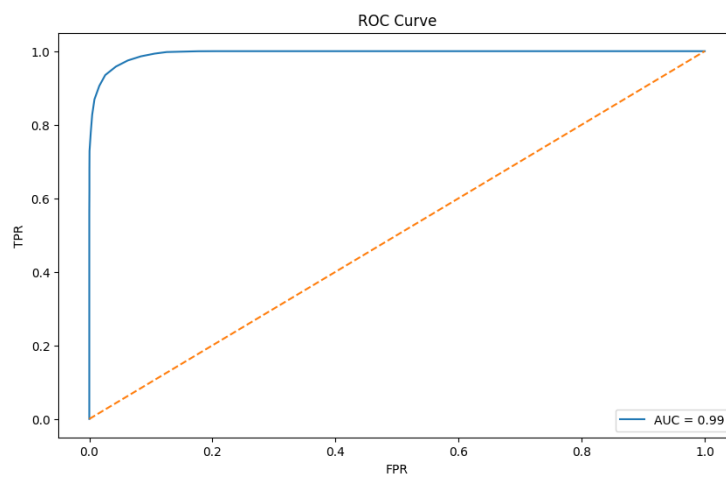


Fig.5 ROC (KNN)

Classification Report:				
	precision	recall	f1-score	support
0	0.96	0.96	0.96	5365
1	0.95	0.96	0.95	4635
accuracy			0.96	10000
macro avg	0.96	0.96	0.96	10000
weighted avg	0.96	0.96	0.96	10000

Fig.6 Classification Report(KNN)

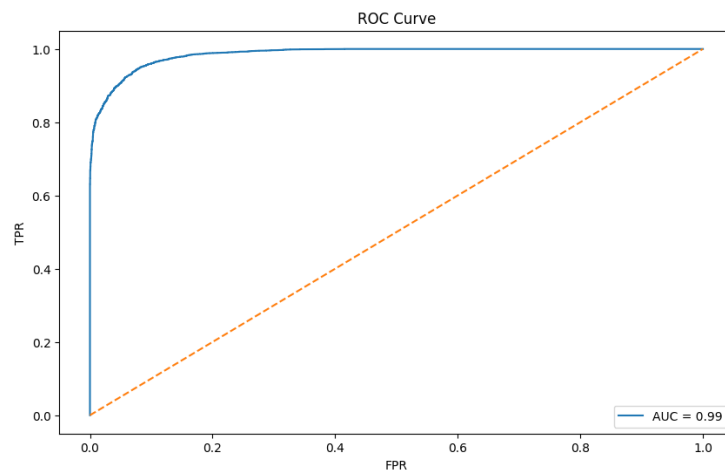


Fig.7 ROC (SVM)

Classification Report:				
	precision	recall	f1-score	support
0	0.94	0.94	0.94	5365
1	0.93	0.93	0.93	4635
accuracy			0.93	10000
macro avg	0.93	0.93	0.93	10000
weighted avg	0.93	0.93	0.93	10000

Fig.8 Classification Report(SVM)

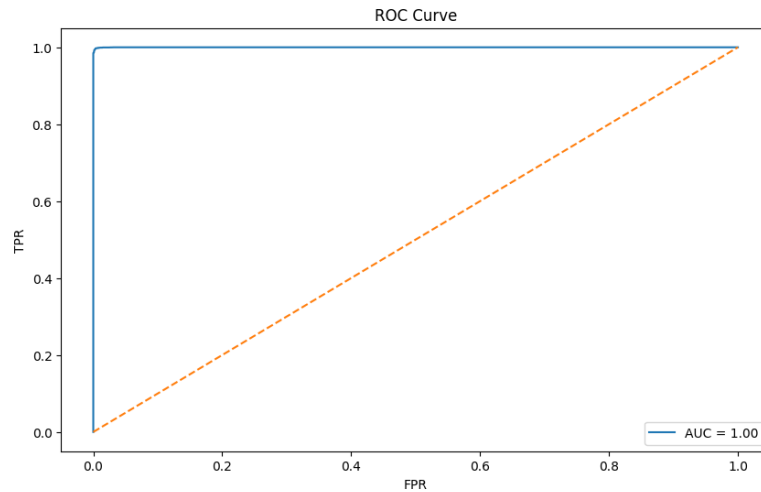


Fig.9 ROC(Extra Trees Classifier)

Classification Report:

	precision	recall	f1-score	support
0	0.996459	0.996645	0.996552	5365
1	0.996116	0.995901	0.996008	4635
accuracy			0.996300	10000
macro avg	0.996287	0.996273	0.996280	10000
weighted avg	0.996300	0.996300	0.996300	10000

Fig.10 Classification Report(Extra Trees Classifier)

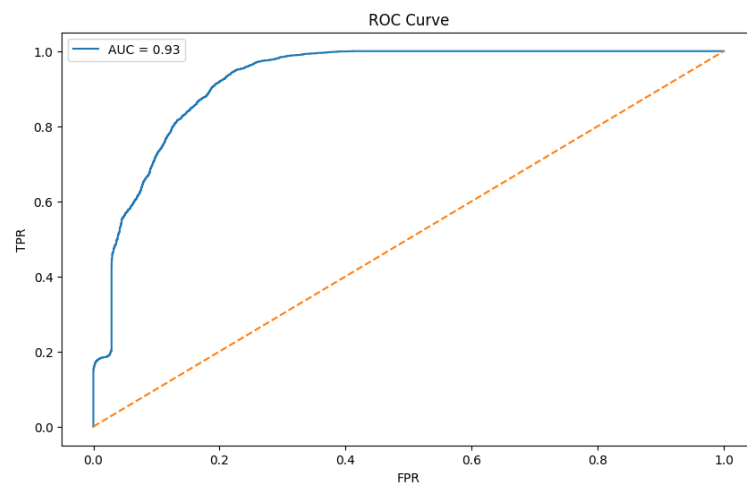


Fig.11 ROC(Gaussian Naive Bayes)

```

Classification Report:
              precision    recall  f1-score   support

     0       0.83         0.88         0.85         5365
     1       0.85         0.79         0.82         4635

 accuracy          0.84         10000
 macro avg         0.84         0.83         0.83         10000
 weighted avg         0.84         0.84         0.84         10000

```

Fig.12 Classification Report(Gaussian Naive Bayes)

6. Conclusion and Future Scope:

Conclusion:

- The project successfully developed a Heart Disease Prediction system using multiple machine learning algorithms.
- Among all models, the Extra Trees Classifier achieved the highest accuracy, making it the best-performing model.
- Important health features like age, lifestyle, and medical history significantly influenced predictions.
- The system can support early detection of heart disease and assist in medical decision-making.

Future Scope:

- Improve accuracy using advanced models (e.g., Deep Learning)
- Use larger and real-time datasets for better reliability
- Deploy as a web/mobile application for easy access
- Integrate with hospital systems or wearable devices
- Add real-time monitoring and alerts for patients

7. References:

1. GeeksforGeeks (ML Basics):
(<https://www.geeksforgeeks.org/data-analysis/exploratory-data-analysis-in-python/>)
2. W3Schools Python:
(https://www.w3schools.com/python/python_ml_linear_regression.asp)
3. Kaggle Dataset:
(<https://www.kaggle.com/datasets/mohamedkhairylbanna/heartdiseasetrain-test>)
4. Class Notes

